



2026

# .NET Aspire na prática

O jeito moderno de construir, executar e observar aplicações distribuídas

**Gustavo Bigardi**

CTO @ Clarus IT · Microsoft MVP





QUEM SOU

# Gustavo Bigardi

CTO @ Clarus IT

Microsoft MVP · 7 vezes consecutivas

[github.com/gustavobigardi](https://github.com/gustavobigardi) · [about.me/gbbigardi](https://about.me/gbbigardi)



**Distribuído em  
produção,  
artesanal no  
laptop.**

Seis containers na nuvem. Três terminais e um  
README na sua máquina.

Quem aqui precisa de mais de um terminal  
para subir o sistema localmente?

# MVP

# Co



# O que vamos ver

**01****A fricção do dia 1**

Como o time sobe o sistema hoje

**02****Aspire em 2026**

O que é, o que mudou e o que não é

**03****Orquestrar**

A Loja antes e depois, ao vivo

**04****Descobrir e observar**

Service discovery, dashboard e traces

**05****Preparar para produção**

publish e deploy sem reescrever

**06****Time e agentes**

Onboarding, MCP e skills

# A fricção do dia 1

Todo time distribuído tem um README de sete passos, três terminais e a frase 'roda de novo'. Esse custo não aparece na fatura: aparece no onboarding e no bug que só acontece na sua máquina.

# O ritual para subir a Loja

```
# README.md, passo a passo
cp .env.example .env
docker-compose up -d
sleep 5 # espera o postgres (às vezes não basta)

# terminal 1
cd recomendacoes && uvicorn main:app --port 8000
# terminal 2
cd Loja.Api && dotnet run # porta 5001
# terminal 3
cd Loja.Web && dotnet run # porta 5010
```

3

terminais abertos

4

portas fixas em 3 arquivos

1

connection string copiada à  
mão

0

correlação entre os logs

**Funciona.** Na maioria das vezes. Para quem já sabe a ordem.

# Onde a fricção cobra

01

## Onboarding

Dias até o primeiro pedido rodar na máquina de quem chegou.

02

## "Na minha máquina funciona"

Config divergente entre dev, CI e produção.

03

## Depuração às cegas

Erro 502 no front, causa no Python, e nada liga uma janela à outra.

04

## Ordem de subida

A API morre se o banco demora. A solução do time: rodar de novo.

### O QUE O TIME PAGA

Tempo

Confiança

Consistência

Foco

# Aspire em 2026

Nasceu como .NET Aspire em 2024. Em novembro de 2025 perdeu o prefixo e virou uma camada de orquestração e observabilidade code-first para qualquer stack. Hoje está na 13.5.

# O que é o Aspire

**01****AppHost**

Um projeto C# ou TypeScript que descreve recursos e dependências. É o mapa do sistema.

**02****Integrações**

Pacotes de hosting (Postgres, Redis, Python...) e de cliente (Npgsql, Redis...) que se ligam pelo nome.

**03****ServiceDefaults**

OpenTelemetry, health checks, service discovery e resiliência em uma chamada, em código seu.

**04****Dashboard e CLI**

Recursos, logs, traces e métricas num lugar. `aspire run`, `publish`, `deploy`, `logs`, `otel`.

**RODA COM****.NET 8, 9 e 10 (11)****Python e JavaScript****Go, Java, Rust, Bun****Containers e executáveis**

# De .NET Aspire a Aspire

mai/2024

## 8.0

GA como workload do .NET. Dashboard, AppHost e as primeiras integrações.

nov/2024

## 9.0

Vira SDK, sem workload. Dashboard cresce, WaitFor, comandos de recurso.

nov/2025

## 13.0

Sem ".NET" no nome. Python e JavaScript de primeira classe. AppHost em um arquivo. Exige .NET 10 SDK.

ago/2026

## 13.5

CLI bundle por padrão, terminal no dashboard, AppHost TypeScript GA, MCP via CLI.

**Por que pulou de 9 para 13?** Quebrou a numeração com o .NET 10 e o ritmo de entregas: cinco releases em nove meses.

**Só o AppHost em C# precisa do SDK do .NET 10.**  
Os serviços podem continuar em .NET 8 ou 9.

# O que o Aspire não é

## Não é

- **Um orquestrador de produção.** Em produção você continua com Container Apps, AKS, App Service ou Compose.
- **Um APM.** O dashboard é de desenvolvimento. Em produção, a telemetria vai por OTLP para Application Insights, Grafana ou o que você já usa.
- **Um framework que invade o código.** ServiceDefaults é um projeto seu; dá para ler e trocar.
- **Só para .NET.** Python, JavaScript, Go, Java e containers entram no mesmo mapa.

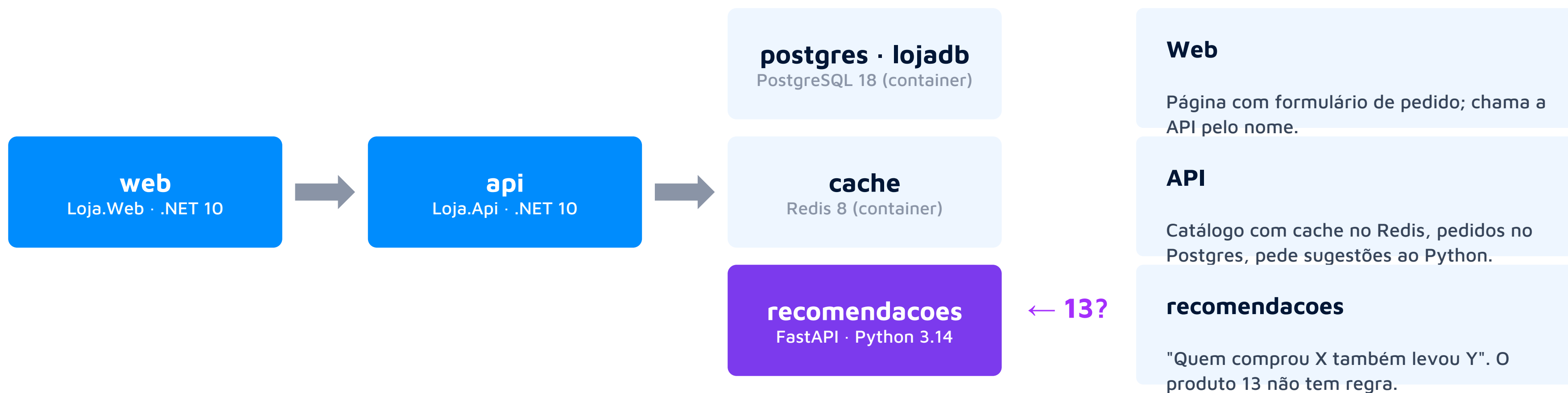
## Comparado ao Docker Compose

|                 | Docker Compose    | Aspire                                        |
|-----------------|-------------------|-----------------------------------------------|
| Escopo          | Containers        | Containers, processos, projetos e nuvem       |
| Configuração    | YAML              | C# ou TypeScript tipado                       |
| Descoberta      | DNS da rede       | Referências injetadas como config             |
| Observabilidade | Logs de container | Logs, traces e métricas correlacionados       |
| Destino         | Docker            | Dev local e publish para Compose, K8s e Azure |

# Orquestrar: antes e depois

A mesma Loja duas vezes: primeiro do jeito que a maioria dos times roda hoje, depois com um AppHost. O código de negócio não muda; muda quem liga as peças.

# A Loja



**Cinco recursos, duas linguagens, dois containers.** Pequena de propósito: cabe na cabeça e na tela.

# Demo 1: a Loja sem Aspire

docker-compose, .env e três terminais

Pedido da caneca falha com 502

Onde está o erro? Em qual janela?

# MVP

# Co



# AppHost: o mapa do sistema em código

```
var builder = DistributedApplication.CreateBuilder(args);

var postgres = builder.AddPostgres("postgres").WithDataVolume().WithPgWeb();
var lojadb = postgres.AddDatabase("lojadb");
var cache = builder.AddRedis("cache").WithRedisInsight();

var recomendacoes = builder.AddUvicornApp("recomendacoes", "../recomendacoes", "main:app")
    .WithHttpEndpoint(env: "PORT").WithHttpHealthCheck("/health");

var api = builder.AddProject<Projects.Loja_Api>("api")
    .WithReference(lojadb).WithReference(cache).WithReference(recomendacoes)
    .WaitFor(lojadb).WaitFor(cache).WaitFor(recomendacoes);

builder.AddProject<Projects.Loja_Web>("web")
    .WithReference(api).WaitFor(api).WithExternalHttpEndpoints();

builder.Build().Run();
```

## AddX = um recurso

Container, projeto .NET, app Python, executável ou serviço na nuvem.

## WithReference = quem fala com quem

Injeta connection string e endpoints como configuração no destino.

## WaitFor = ordem de subida

Segura o recurso até a dependência passar no health check.

# Demo 2: aspire run

Um comando, nove recursos, um dashboard

WaitFor, health checks, pgweb e RedisInsight  
de brinde

A caneca de novo: o trace mostra onde doeu

# MVP

# Code



# O que mudou no código

## ANTES

```
builder.Services.AddDbContext<LojaDb>(o =>
    o.UseNpgsql(config.GetConnectionString("lojadb")));
builder.Services.AddStackExchangeRedisCache(o =>
    o.Configuration = config["Redis"]);
client.BaseAddress = new Uri(config["Recomendacoes:Url"]!);

// appsettings.json
"lojadb": "Host=localhost;Port=5432;...;Password=postgres"
"Redis": "localhost:6379"
"Recomendacoes": { "Url": "http://localhost:8000" }
```

## DEPOIS

```
builder.AddServiceDefaults();
builder.AddNpgsqlDbContext<LojaDb>("lojadb");
builder.AddRedisDistributedCache("cache");
client.BaseAddress = new Uri("https+http://recomendacoes");

app.MapDefaultEndpoints(); // /health e /alive

// appsettings.json
// (nenhuma connection string, nenhuma porta)
```

**0** connection strings no repositório

**3** arquivos por serviço: csproj, Program.cs, appsettings

# WithReference não é WaitFor

## WithReference

Injeta connection string e endpoints como variáveis de ambiente no destino. Diz quem fala com quem. Não espera ninguém.

## WaitFor

Segura a subida até a dependência ficar Healthy pelo health check do AppHost. Diz a ordem.

## WaitForCompletion

Para jobs: espera terminar com sucesso. Migração de banco, seed, geração de código.

## Health check do AppHost

"A dependência está pronta?" Container Postgres, Redis e Python respondendo. É o que o WaitFor consulta.

## Health check do serviço

"Posso receber tráfego?" /health (pronto) e /alive (vivo), criados pelo MapDefaultEndpoints. Kubernetes e Container Apps usam isso.

# Descobrir e observar

Nenhum serviço da Loja sabe a porta do outro. E nenhum instalou agente de APM. Os dois problemas se resolvem com variáveis de ambiente que o AppHost injeta.

# Service discovery: nomes, não portas

```
// no serviço, só o nome lógico
client.BaseAddress = new Uri("https+http://api");
```

```
# o que o AppHost injetou no processo (aspire run)
services__api__http__0      = http://localhost:5201
services__api__https__0     = https://localhost:7107
ConnectionStrings__lojadb   = Host=localhost;Port=52934;Username=postgres;...
ConnectionStrings__cache    = localhost:52932,password=...
OTEL_EXPORTER_OTLP_ENDPOINT = (endpoint gRPC do dashboard)
```

## Local

localhost e portas aleatórias; o resolvedor lê as variáveis.

## Docker Compose

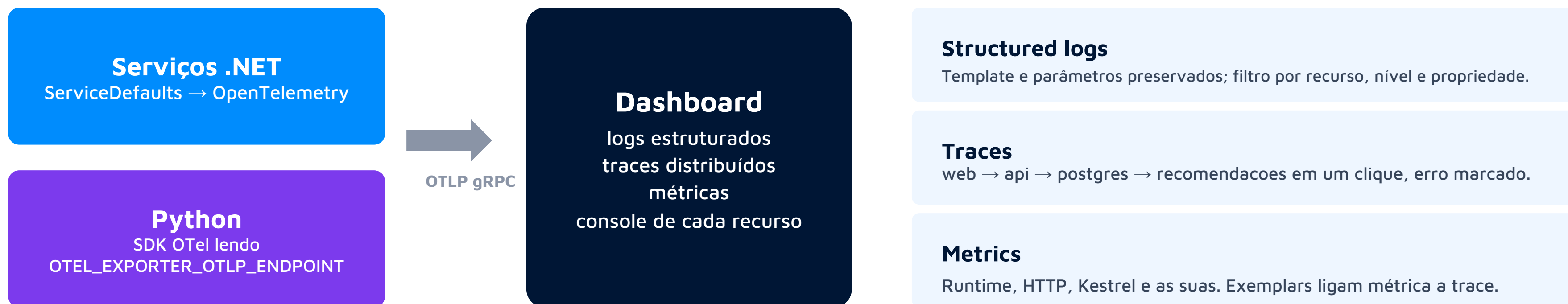
http://api:8080 na rede aspire, gerado no publish.

## Container Apps e Kubernetes

DNS interno da plataforma. O mesmo código, sem if de ambiente.

**https+http://** = prefira HTTPS, caia para HTTP em desenvolvimento. Python e Node leem as mesmas variáveis.

# Observabilidade sem instalar nada



**O que o AppHost injeta:** OTEL\_SERVICE\_NAME, OTEL\_EXPORTER\_OTLP\_ENDPOINT, OTEL\_EXPORTER\_OTLP\_PROTOCOL e OTEL\_EXPORTER\_OTLP\_HEADERS.

**O que continua sendo seu:** instrumentar. Em Python foram 40 linhas de telemetry.py; em .NET, zero.

# O que o padrão faz por você (e contra você)

## Resiliência padrão do ServiceDefaults

- Retry: 3 tentativas com backoff
- Timeout: 10 s por tentativa, 30 s no total
- Circuit breaker e limite de concorrência
- Aplicado a todo HttpClient do serviço

## 3 canecas

**em 30 segundos.** O retry padrão retenta POST: o front repetiu o pedido três vezes e a API esperou o Python três vezes. Pedido duplicado e usuário olhando o spinner.

```
// Loja.Web: só retenta GET
.AddStandardResilienceHandler(o =>
    o.Retry.ShouldHandle = args => ValueTask.FromResult(
        args.Outcome.Result?.RequestMessage?.Method
            == HttpMethod.Get
            && HttpClientResiliencePredicates
                .IsTransient(args.Outcome)));

// Loja.Api: falhe rápido na recomendação
o.Retry.MaxRetryAttempts = 1;
o.AttemptTimeout.Timeout = TimeSpan.FromSeconds(3);
```

**Padrão bom é padrão que você conhece.** O trace mostrou 61 spans e 30 s. Sem ele, eu teria culpado o Python.

# Preparar para produção

O AppHost já sabe o que o sistema precisa. Publicar é traduzir esse mapa para o destino: Compose, Kubernetes ou Azure, sem escrever YAML na mão.

# Mesmo AppHost, outros destinos

## Docker Compose

AddDockerComposeEnvironment:  
compose, .env e Dockerfiles  
gerados



## Kubernetes e AKS

Helm chart, Ingress e Gateway API;  
AKS com  
AddAzureKubernetesEnvironment



## Azure Container Apps

AddAzureContainerAppEnvironment:  
Bicep, ACR, identidade e  
dashboard

### aspire publish gera

- docker-compose.yaml ou Bicep, conforme o destino
- Dockerfiles, inclusive do Python
- .env com os parâmetros que faltam
- Nenhuma imagem ainda: é rápido e revisável

### aspire deploy faz

- Constrói e envia as imagens
- Provisiona os recursos do destino
- Guarda estado entre execuções
- aspire destroy desfaz tudo
- App Service em preview; AWS no roadmap

# Demo 3: aspire publish

Docker Compose gerado em 2 segundos  
Dockerfile do Python escrito pelo Aspire  
pgweb e RedisInsight somem: são só de dev

# MVP

# Code



# E no Azure?

## O que aspire deploy cria no Container Apps

- Azure Container Registry e as imagens da Loja
- Container Apps Environment com Log Analytics
- Identidade gerenciada com permissão de pull
- Storage Account e Azure Files para o volume do Postgres
- Dashboard do Aspire protegido por RBAC (Contributor ou Owner)
- Cinco container apps: web (externo), api, recomendacoes, postgres, cache

## Postgres e Redis em container é demo

Em produção: `AddAzurePostgresFlexibleServer(...).RunAsContainer()`.  
Container no laptop, PaaS na nuvem, mesmo AppHost.

## O dashboard no ACA é para olhar, não para guardar

Telemetria em produção vai por OTLP para Application Insights, Grafana ou o seu coletor. O dashboard é a lupa, não o arquivo.

`Deploy__Target=aca aspire deploy` pergunta subscription e região uma vez e guarda o estado. `aspire destroy` apaga o resource group inteiro.

# Time e agentes

Aspire é sobre o time: quem entra roda o sistema no primeiro dia, e quem depura tem o mesmo dado que o agente de IA vai ler.

# Um dia 1 diferente

**01****aspire run**

Clonou, rodou. Sem README de sete passos, sem sleep 5.

**02****aspire init + aspireify**

App existente ganha AppHost com um comando; a skill orienta o agente a terminar a fiação.

**03****--isolated**

Dois AppHosts na mesma máquina, com portas e segredos separados.

**04****AppHost em TypeScript**

GA desde a 13.4. O time de front escreve o mapa na língua dele.

**TAMBÉM AJUDA****Aspire.Hosting.Testing****aspire update****Dev tunnels****Comandos de recurso**

# Demo 4: um agente lendo o dashboard

aspire agent mcp expõe recursos, logs e traces

GitHub Copilot: "por que o pedido da caneca falha?"

A resposta vem do trace, não do achismo

# MVP

# Coop



# Como o agente vê o sistema

## Ferramentas do MCP (aspire agent mcp)

`list_resources` estado, saúde, endpoints e comandos  
`list_structured_logs` filtrados por recurso  
`list_traces` e `list_trace_structured_logs`  
`list_console_logs`  
`execute_resource_command` start, stop, restart  
`search_docs`, `get_doc`, `list_integrations`  
`doctor` diagnóstico do ambiente

## Skills (aspire agent init)

aspire, aspire-init, aspire-orchestration, aspire-monitoring, aspire-deployment e aspireify: markdown que ensina o agente a usar o CLI e o AppHost. Vão para `.agents/skills` e para `.vscode/mcp.json`.

## CLI feito para agentes

```
aspire logs api --search KeyError
aspire otel traces recomendacoes
aspire describe --format Json
```

Mesmo dado do dashboard, em texto. O agente lê o que você leria.

Orquestração local  
não é luxo. É onde  
o time passa o dia.

Se o sistema é distribuído em produção,  
precisa ser simples de rodar no laptop.

O AppHost é o contrato entre os dois mundos.

# MVP

# Coop



# Leve para casa

## SEGUNDA-FEIRA

**aspire init num app que você já tem e rode aspire run**

## PRÓXIMA FEATURE

**WithReference + WaitFor + health check; nenhuma connection string no repositório**

## TODO SPRINT

**Abra o dashboard antes de abrir o console; leve o trace para a retro**



**[github.com/gustavobigardi/  
aspire-na-pratica](https://github.com/gustavobigardi/aspire-na-pratica)**

**A Loja antes e depois, AppHost, roteiro das  
demos e referências**

# Siga MVP Conf nas redes sociais



LINKEDIN



@MVPCONF

# Obrigado!

**Gustavo Bigardi**

[github.com/gustavobigardi](https://github.com/gustavobigardi) · [about.me/gbbigardi](https://about.me/gbbigardi)